

What Your Pipeline Reports Is Not What It Ran: Silent Failure in an ARC-AGI-2 Solver

ARC Prize 2026 — Paper Track Writeup

Abstract

Pipelines report the work they intend to do, not the work they did. We instrumented a test-time-training (TTT) ARC-AGI-2 solver and found that of 102 tasks routed to adaptation, **16** executed a single optimizer step: **the stage spent 7,266 seconds to perform 21 steps, and the 84 tasks that never trained were recorded as evidence that adaptation did not help.** The cause was mundane — a memory allocation — but the reporting made it invisible. Accuracy cannot distinguish "the stage failed" from "the stage ran and did not help", because graceful degradation under a fixed output contract makes both identical. We place this beside two sibling failures from the same campaign, and recommend one counter: **report executed optimizer steps, not intended ones.**

1. Introduction

ARC-AGI-2 asks for a rule inferred from a few demonstration pairs. Because each task *is* a tiny training set, test-time training is the dominant approach [1]. Our system follows that lineage — a 4B open model, per-task LoRA, thresholded beam search — the configuration NVARC won 2025 with [6]. The premise is that the model is adapted before it predicts. This paper is about the gap between that premise and what a run does.

Accuracy cannot close that gap. It is the product of *generating* the right answer and *selecting* it; when it is low the natural inference is that the model is not good enough. We made that inference, then measured, and found something simpler: the adaptation had barely run, and every number we had said otherwise.

2. Prior work

ARC-AGI-2 is harder and less brute-forcible than ARC-AGI-1 [8, 2]; the 2025 top score was 24.03% [9]. Solver families include DSL search, neuro-symbolic induction [4], recursive networks, and TTT-based transduction, which dominates — the 2024 report states all top LLM-based entries use it [1]. NVARC's winner is Qwen3-4B with LoRA [6], so our system is a scaled-down instance of an established recipe, not a new solver.

Memory-bound TTT-for-ARC is well documented: 16 GB P100 [3], L4×4 [10], rank-32 LoRA at batch 1 [5]. **We found no prior report of a masked adaptation failure**, which is the claim we make.

The generation/selection decomposition is *not* ours and we cite it rather than claim it: ARChitects' "coverage" is described as "an upper bound for the performance of the selection algorithms" [5, Fig. 4]; Li et al. publish "Sample+Oracle" [4, Fig. 8]; Moghe & Chin make the split their headline [7]. Split-distribution mismatch is likewise published — 6.9% of train versus 40.8% of eval tasks require multiple answers [11], against our measured 7.1% and 40.8%.

3. Approach

System. Qwen3-4B fine-tuned to a 16-token grid vocabulary (3.634B parameters); per-task LoRA (rank 256, rsLoRA, on all projections plus `embed_tokens` and `lm_head`); cosine schedule; a constrained depth-first beam search over the next-token distribution; cascade scheduling (cheap sweep, then TTT on the weakest tasks, then symbolic backfill). Every emitted grid is validated for shape and colour range.

Instrumentation. `ttt_steps` is incremented *inside* the optimizer loop, so it counts steps

that ran. Pool recall asks whether the ground-truth grid appears anywhere in a task's candidate pool — borrowed practice (§2), used only to locate the loss.

Two measurement bugs we had to fix first, both of which favoured the hypothesis under test. The pool-recall aggregate was accumulated per **stage** rather than per task, publishing a denominator of 16 where 11 test inputs existed. And reports predating the instrumentation lack the field entirely, which reads as "0 in pool" and manufactures the confirmation being looked for. A diagnostic whose failure mode is **confirms what you hoped** is worse than no diagnostic.

4. Results

4.1 We are generation-bound, with zero selection headroom

Pool recall was zero in all four instrumented runs: **0 of 35 (configuration, input)** measurements. **Those are** 8 unique evaluation tasks and 11 unique test inputs **re-measured under** three configurations — not 35 independent trials, and we do not report them as such. A zero count over 11 independent inputs bounds the true rate at $\leq 23.8\%$ (**95%**); we give the bound rather than the flattering point estimate. Pools held **4–29 distinct** candidates, so "the pool was too small" is ruled out.

4.2 Search width is not the lever

Three paired configurations, identical tasks, budget, augmentations and seed. Halving the token probability threshold ($0.2 \rightarrow 0.1$) and doubling both beam breadth ($3 \rightarrow 4$) and node budget ($6,000 \rightarrow 12,000$) changed nothing: 0/8 solved and 0.0% recall in every configuration, at 30% more time per task ($203 \rightarrow 263$ s).

4.3 The adaptation barely executed

In the 240-task run, Stage B received **102** tasks; **16** executed at least one optimizer step; **84** were recorded ``B_ttt_no_gain`` with **zero** steps; the run performed **21 optimizer steps in** total **while the stage consumed** 7,266 seconds. **The mechanism is an out-of-memory failure on** step 0.

There are two masking paths. Without a circuit breaker each task fails independently, the pipeline records "no gain", and the run continues. With a breaker, one failure disables the mechanism run-wide — the **reasonable** engineering choice, which escalates a single task's failure into a systematically unadapted submission. The failure is structural rather than incidental: adaptation is an optional refinement inside a pipeline that must emit a submission regardless, so a resource failure degrades gracefully into "unadapted model". **Graceful degradation is what makes it** silent.

4.4 The accelerator was wrong, and nothing said so

Every experiment above ran on one Tesla T4 (14.56 GiB). The competition offers ``NvidiaL4``: four L4s, 88 GiB. Our own compatibility guard **reported the L4 as unusable**, because it tested membership in ``torch.cuda.get_arch_list()``, which omits ``sm_89``; real work succeeds on an L4. So the memory pressure in §4.3 was self-inflicted. The lesson is not the memory but the reporting: a wrong configuration produced a run whose error list was empty and whose output read as a capability result. We did not notice for a week, which is the same silence this paper is about.

4.5 The local validation loop was invalid

The public test file is not a holdout: all 240 tasks are byte-identical to training tasks, their

answers ship alongside them, and overlap with the evaluation split is zero. It also under-represents multi-answer tasks (7.1% versus 40.8%), so a submission builder that is correct locally is malformed on the hidden set. The structural half reproduces [11]; we claim no priority.

5. Discussion

What is new. Pool recall is not ours, the generation-bound finding is a replication, search width is confirmatory, distribution mismatch is a reproduction. **The masked adaptation failure is** the one claim we make.

The safeguard. Report executed optimizer steps per task. One counter distinguishes "did not help" from "did not happen". Two corollaries: make degradation per-task rather than global, and never report a stage's null result when the stage did not run — recording 84 zero-step tasks as `B\_ttt\_no\_gain` was the most misleading thing our own reporting did. The counter is released as a dependency-free module with an 11-check self-test, so the recommendation is adoptable rather than merely advisable.

What would raise the score, identified but not validated: make adaptation fit by bounding the training-sequence context and treating OOM as per-task degradation, on the larger accelerator.

6. Conclusion

We found that the part of our solver we believed in most was not running, and that every number we had said otherwise. The contribution is the failure mode and its safeguard: optional learned refinement stages fail silently by construction, and the fix is one counter.

Appendix A — Artefact and submission linkage

Solver, diagnostic tooling, raw run reports and kernel logs are public under MIT-0. Every number here is re-derived by `tools/evidence\_ledger.py`, which prints each cited value beside the file it came from; 90 checks require no GPU and all pass, including regression checks for both bugs in §3. The linked Kaggle notebook writes `/kaggle/working/submission.json` with internet disabled. The 240-task run cost 37,963 s on one T4 at 14.12 GiB peak. Leaderboard score: **0.42%** (public, 120-task semi-private set) — a real scored entry, which the rules require for eligibility and which they explicitly do not require to be high.

References

1. ARC Prize 2024 Technical Report. arXiv:2412.04604
2. Chollet. On the Measure of Intelligence. arXiv:1911.01547
3. Cole & Osman. arXiv:2506.14276
4. Li et al. Combining Induction and Transduction. arXiv:2411.02272
5. ARChitects. Product of Experts. arXiv:2505.07859
6. NVARC. github.com/1ytic/NVARC
7. Moghe & Chin. arXiv:2607.06764
8. ARC-AGI-2. arXiv:2505.11831
9. ARC Prize 2025 Technical Report. arXiv:2601.10904
10. ARC Prize 2025 archive. arcprize.org/competitions/2025/archive
11. Habr, split-distribution analysis. habr.com/ru/articles/1071730/